

## **Title of the Project:** Real-Time Audio Signal Processing and Efficient Long-Sequence Convolution

Design and implement a MATLAB system for long sequence convolution using both the Overlap-Add and Overlap-Save methods. Utilize real-time input signals such as speech, audio, or biomedical data. Perform a comparative analysis of the computational efficiency and performance of the two techniques. (Q. No.1 is common to all the batches).

### **1. Introduction**

In this experiment, we perform long sequence convolution using two efficient methods — Overlap-Add (OLA) and Overlap-Save (OLS). Instead of directly applying convolution, which can be slow for long signals, these methods divide the input into smaller blocks and process them efficiently using Fast Fourier Transform (FFT). We use a real-time audio signal to analyze and compare the performance of both techniques.

### **2. Objective**

To design and implement a MATLAB system that performs long sequence convolution using both the Overlap-Add and Overlap-Save methods on real-time signals like speech or audio, and compare their computational efficiency and output performance.

### **3. Theory**

#### **3.1 Linear Convolution**

Linear convolution gives the output of an LTI system for any input sequence. If the input signal is  $x[n]$  and the impulse response is  $h[n]$ , then the linear convolution is:

$$y[n] = \sum x[k] * h[n - k]$$

This operation becomes computationally expensive for long sequences, which is why methods like Overlap-Add and Overlap-Save are used.

#### **3.2 Overlap-Add Method**

The Overlap-Add method breaks the long input signal into smaller blocks. Each block is convolved with the impulse response using FFT, and then the overlapping parts are added together.

If  $x[n]$  is divided into sections of length  $L$  and  $h[n]$  has length  $M$ , then the circular convolution of each block with  $h[n]$  gives a segment of length  $(L+M-1)$ . The overlapping regions between successive blocks are summed to obtain the final linear convolution output.

### 3.3 Overlap-Save Method

The Overlap-Save method also divides the input into blocks, but it keeps overlapping samples from previous segments. Each block of length  $(L+M-1)$  is processed using circular convolution with  $h[n]$ . After convolution, the first  $(M-1)$  samples are discarded, and the remaining  $L$  samples are saved as output.

This method avoids explicit overlap addition and often performs faster in practice.

## 4. MATLAB Code and Block-Wise Explanation

### Block 1: Input Signal Acquisition and Preprocessing

```
[file, path] = uigetfile({'*.wav;*.mp3'});
space_beat = fullfile(path, file);
[x, Fs] = audioread(space_beat);
x = x(1:min(end, Fs*30), :);
if size(x, 2) == 2
    x = mean(x, 2);
end
x = x(:)';
```

#### Explanation:

This block allows the user to select an audio file (or enter manual data in manual mode). It reads and preprocesses the signal for further use. Stereo audio is converted to mono for simplicity.

### Block 2: Impulse Response Selection

```
disp("Select an option:");
disp("1. Built-in Impulse Response");
disp("2. Manual Input");
opt1 = input('Enter your choice: ');
if opt1 == 1
    h = [1 0.8 0.6 0.4 0.2]; % Example built-in impulse (reverb)
else
    h = input('Enter impulse response h(n): ');
end
```

#### Explanation:

This section lets the user select between a pre-defined impulse response (reverb) or manually enter their own  $h(n)$ .

### Block 3: Block Size Selection

```
Lx = length(x);
M = length(h);
L = input('Enter block length (L < Lx): ');
N = L + M - 1;
```

#### Explanation:

Here, the user selects the block size  $L$  for processing. The convolution will use FFT blocks of length  $N = L + M - 1$ .

## Block 4: Overlap-Add Method Implementation

```
tic;
Y_add = zeros(1, Lx + M - 1);
h1 = [h zeros(1, N - M)];
for i = 1:floor(Lx/L)
    si = (i-1)*L + 1;
    ei = si + L - 1;
    segment = x(si:ei);
    segment = [segment zeros(1, N - L)];
    Y = ifft(fft(segment).*fft(h1));
    Y_add(si:si+N-1) = Y_add(si:si+N-1) + Y;
end
tOA = toc;
```

### Explanation:

This block implements the Overlap-Add method using FFT for block-wise convolution and adds overlapping segments.

## Block 5: Overlap-Save Method Implementation

```
tic;
Y_save = zeros(1, Lx + M - 1);
x_pad = [zeros(1, M-1) x];
for i = 1:floor(Lx/L)
    si = (i-1)*L + 1;
    segment = x_pad(si:si+N-1);
    Y = ifft(fft(segment).*fft(h1));
    Y_save(si:si+L-1) = Y(M:end);
end
tOS = toc;
```

### Explanation:

This block uses the Overlap-Save method, where overlapping samples from previous blocks are reused and the first (M-1) samples of each block are discarded.

## Block 6: Computational Performance Comparison

```
disp('Time taken by Overlap-Add:'), disp(tOA)
disp('Time taken by Overlap-Save:'), disp(tOS)
if tOA < tOS
    disp('Overlap-Add is faster');
else
    disp('Overlap-Save is faster');
end
```

### Explanation:

This section measures and compares the computational time for both methods, helping analyze efficiency.

## Block 7: Output Visualization

```
t = (0:length(x)-1)/Fs;
t1 = (0:length(Y_add)-1)/Fs;
figure;
subplot(3,1,1); plot(t, x); title('Original Signal');
subplot(3,1,2); stem(h); title('Impulse Response');
subplot(3,1,3); plot(t1, Y_add); title('Output Signal (Reverberated)');
```

### Explanation:

This block plots the input audio signal, impulse response, and final output for visual comparison.

### MATLAB Implementation

```
clc;
clear;
close all;

disp('=====');
disp('  LINEAR CONVOLUTION USING OVERLAP-ADD / OVERLAP-SAVE');
disp('=====');

%% ===== Step 1: Input Signal =====
disp('Choose Input Signal Type:');
disp('  1. Load Audio File (.wav or .mp3)');
disp('  2. Manual Entry');
inputChoice = input('Enter your choice (1 or 2): ');

if inputChoice == 1
    [fileName, filePath] = uigetfile({'*.wav;*.mp3'}, 'Select Audio File');
    if isequal(fileName, 0)
        error('No file selected.');
```

end

fullPath = fullfile(filePath, fileName);  
[inputSignal, Fs] = audioread(fullPath);

inputSignal = inputSignal(1:min(end, Fs\*30), :);  
if size(inputSignal,2) == 2  
 inputSignal = mean(inputSignal,2);  
end  
inputSignal = inputSignal(:)'; % row vector  
inputLength = length(inputSignal);  
timeAxisInput = (0:inputLength-1)/Fs;

figure;  
plot(timeAxisInput, inputSignal, 'b');  
xlabel('Time (s)');  
ylabel('Amplitude');  
title('Original Audio Signal');  
grid on;

else  
 inputSignal = input('Enter input sequence x[n]: ');  
Fs = 1000; % dummy for manual signals  
inputLength = length(inputSignal);

end

```

%% ===== Step 2: Impulse Response =====
if inputChoice == 1
    disp('Select Impulse Response Type:');
    disp(' 1. Built-in Reverb');
    disp(' 2. Manual Input');
    hChoice = input('Enter option: ');
    if hChoice == 1
        h = [0.9, zeros(1,1000), 0.6, zeros(1,1200), 0.3];
    else
        h = input('Enter impulse response h[n]: ');
    end
else
    h = input('Enter impulse response h[n]: ');
end
M = length(h);

%% ===== Step 3: Block Length =====
L = inputLength + 1;
while L > inputLength
    L = input(['Enter block length L (< ', num2str(inputLength), '): ']);
    if L > inputLength
        disp('Block length must be smaller than input signal length. ');
    end
end

N = L + M - 1;          % FFT/cconv length for each block
totalConvLength = inputLength + M - 1;
nBlocks = ceil(inputLength / L);

hPadded = [h, zeros(1, N-M)];

%% ===== Step 4: Overlap-Add =====
tic;
Y_add = zeros(1, totalConvLength);
for k = 1:nBlocks
    si = (k-1)*L + 1;
    ei = min(si+L-1, inputLength);
    currentBlock = inputSignal(si:ei);
    currentBlock = [currentBlock, zeros(1, N - length(currentBlock))];
    Y = cconv(currentBlock, hPadded, N);
    outEnd = min(si+N-1, totalConvLength);
    Y_add(si:outEnd) = Y_add(si:outEnd) + Y(1:(outEnd-si+1));
end
timeOLA = toc;

%% ===== Step 5: Overlap-Save =====
tic;
Y_save = zeros(1, totalConvLength);
for k = 1:nBlocks
    si = (k-1)*L + 1;
    ei = min(si+L-1, inputLength);
    block = inputSignal(si:ei);

    if k == 1
        segment = [zeros(1, M-1), block];
    else
        overlap = inputSignal(max(si-M+1,1):si-1);
        if length(overlap) < M-1
            overlap = [zeros(1, M-1-length(overlap)), overlap];
        end
        segment = [overlap, block];
    end

    segment = [segment, zeros(1, N - length(segment))];

```

```

Yseg = cconv(segment, hPadded, N);

outStart = si;
outEnd = min(si+L-1, totalConvLength);
Y_save(outStart:outEnd) = Yseg(M:M+outEnd-outStart);
end
timeOLS = toc;

%% ===== Step 6: Efficiency Comparison =====
disp('=====');
disp('Efficiencies of Overlap Add and Overlap Save (seconds):');
disp(['Overlap-Add: ', num2str(timeOLA)]);
disp(['Overlap-Save: ', num2str(timeOLS)]);
if timeOLA < timeOLS
    disp('Result: Overlap Add is faster.');
```

```
elseif timeOLS < timeOLA
    disp('Result: Overlap Save is faster.');
```

```
else
    disp('Result: Both methods are equally fast.');
```

```
end
disp('=====');
```

```
%% ===== Step 7: Output Selection and Display =====
opt = 0;
while ~ismember(opt, [1 2])
    disp('Select output:');
    disp('1. Overlap-Add Output');
    disp('2. Overlap-Save Output');
    opt = input('Enter option: ');
    if opt == 1
        finalOutput = Y_add;
    elseif opt == 2
        finalOutput = Y_save;
    else
        disp('Enter a valid option');
```

```
    end
end

timeAxisOutput = (0:length(finalOutput)-1)/Fs;
figure;
plot(timeAxisOutput, finalOutput);
xlabel('Time (s)');
ylabel('Amplitude');
title('Modified Signal');
grid on;

%% ===== Step 8: Plot Results =====

figure;
subplot(3,1,1); plot(inputSignal); title('Input Signal');
```

```
subplot(3,1,2); plot(h); title('Impulse Response');
```

```
subplot(3,1,3); plot(finalOutput); title('Convolved Output');
```

```
xlabel('Samples'); ylabel('Amplitude');
```

```
%% ===== Step 9: Manual Entry Output Display =====
opt = 0;
while ~ismember(opt, [1 2])
    disp('Select output to display:');
    disp('1. Overlap-Add Output');
```

```
disp('2. Overlap-Save Output');
```

```
opt = input('Enter option: ');
end
if opt == 1
    finalOutput = Y_add;
```

```
else
```

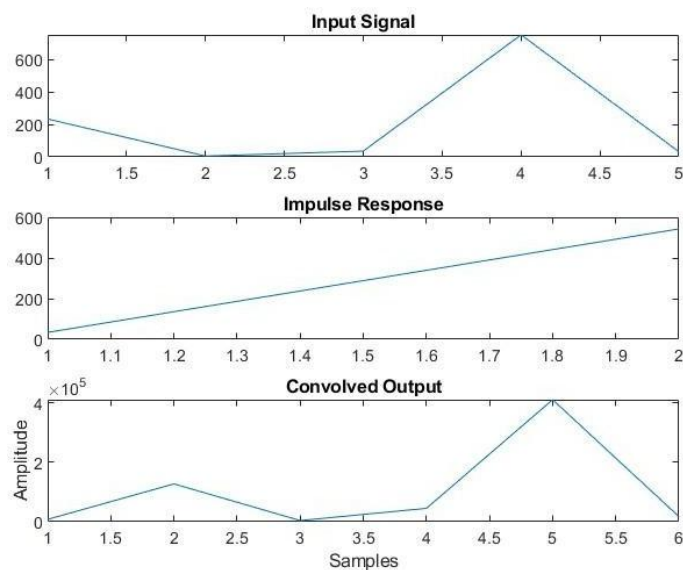
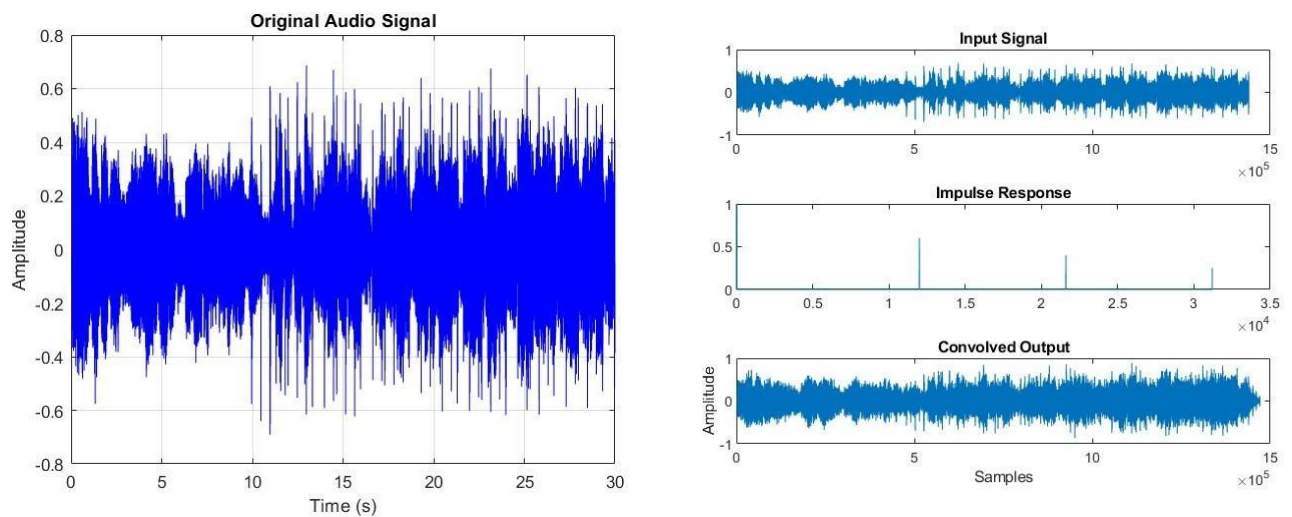
```

        finalOutput = Y_save;
    end

    %% ===== Step 10: Playback for Audio Input Only =====
    if inputChoice == 1
        while true
            inp = input('Play modified:1, Play original:2, End:3: ');
            if inp == 1
                sound(finalOutput, Fs);
            elseif inp == 2
                sound(inputSignal, Fs);
            else
                break;
            end
        end
    else
        disp('Input Signal:'); disp(inputSignal);
        disp('Impulse Response:'); disp(h);
        disp('Convolved Output:'); disp(finalOutput);
    end
end

```

**Figure:**



## 5. Results and Conclusion

The MATLAB system successfully performed convolution on real-time audio data using both Overlap-Add and Overlap-Save methods. The plotted results show how the impulse response modifies the signal (reverb effect). From the timing analysis, one can observe which method performs faster depending on the block size and data length.

### Figures:

```
Command Window

=====
LINEAR CONVOLUTION USING OVERLAP-ADD / OVERLAP-SAVE
=====

Choose Input Signal Type:
  1. Load Audio File (.wav or .mp3)
  2. Manual Entry
Enter your choice (1 or 2): 1
Select Impulse Response Type:
  1. Built-in Reverb
  2. Manual Input
Enter option: 1
Enter block length L (< 1440000): 100
=====

Efficiencies of Overlap Add and Overlap Save (seconds):
Overlap-Add: 32.375
Overlap-Save: 32.2035
Result: Overlap Save is faster.
=====

Select output:
  1. Overlap-Add Output
  2. Overlap-Save Output
Enter option: 2
Play modified:1, Play original:2, End:3: 1
Playing modified audio...
Play modified:1, Play original:2, End:3: 2
Playing original audio...
Play modified:1, Play original:2, End:3: 3
Ending session.

fx >>
```

```
=====
LINEAR CONVOLUTION USING OVERLAP-ADD / OVERLAP-SAVE
=====

Choose Input Signal Type:
  1. Load Audio File (.wav or .mp3)
  2. Manual Entry
Enter your choice (1 or 2): 2
Enter input sequence x[n]: [342 5657 43 345 987 34 43 78.30 -2134]
Enter impulse response h[n]: [0.98 23 54]
Enter block length L (< 9): 4
=====

Efficiencies of Overlap Add and Overlap Save (seconds):
Overlap-Add: 0.0044688
Overlap-Save: 0.0016233
Result: Overlap Save is faster.
=====

Select output:
  1. Overlap-Add Output
  2. Overlap-Save Output
Enter option: 1
Select output to display:
  1. Overlap-Add Output
  2. Overlap-Save Output
Enter option: 1
Input Signal:
  1.0e+03 *

    0.3420    5.6570    0.0430    0.3450    0.9870    0.0340    0.0430    0.0783   -2.1340

Impulse Response:
    0.9800   23.0000   54.0000

Convolved Output:
  1.0e+05 *

    0.0034    0.1341    1.4862    3.0681    0.1122    0.4136    0.5412    0.0290    0.0203   -0.4485   -1.1524
```

**Conclusion:**

Both methods give the same output as linear convolution. Overlap-Save often performs slightly faster due to reduced overlap handling. These techniques are widely used in real-time DSP applications like audio effects, biomedical signal enhancement, and communication systems.